Part 5 Section 1: Markov Chain Monte Carlo

Aaron Robotham

Libraries needed for this chapter: magicaxis, DiagrammeR, LaplacesDemon.

```
library(magicaxis, quietly=TRUE)
library(DiagrammeR, quietly=TRUE)
library(LaplacesDemon, quietly=TRUE)
```

So everybody running this chapter gets the same outputs, we should first set the seed.

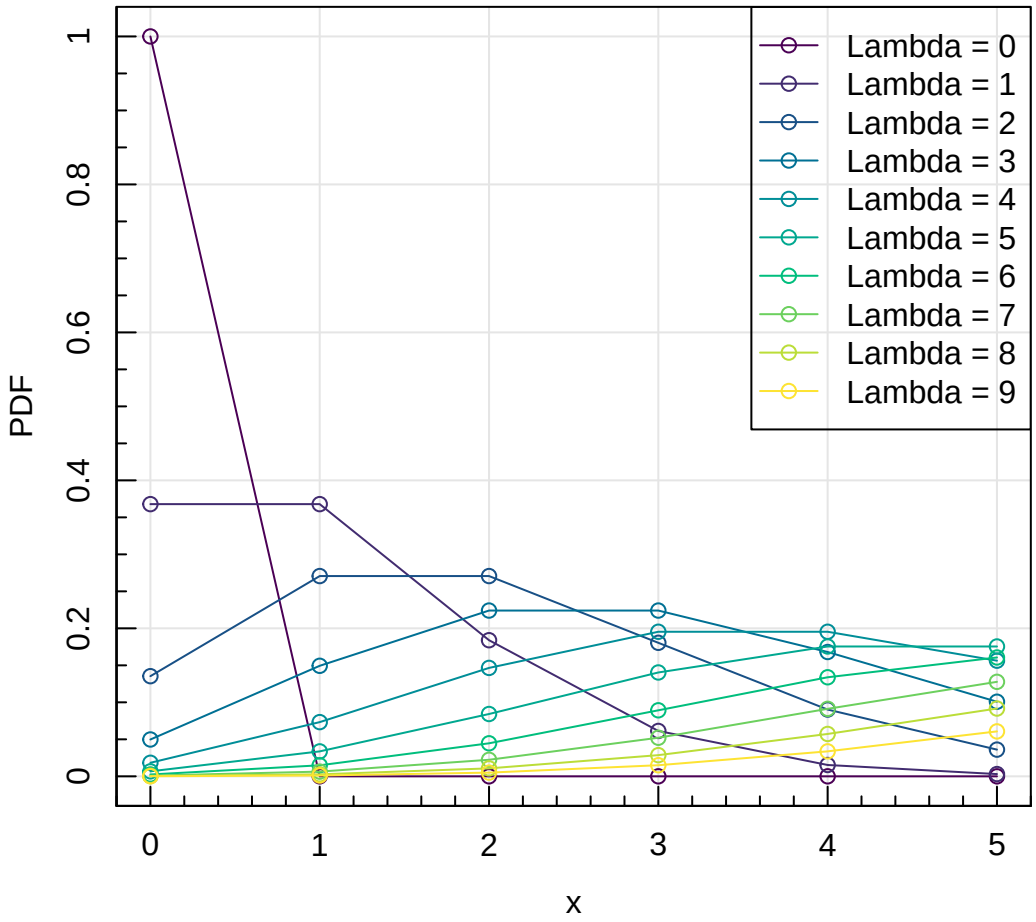
```
set.seed(1)
```

Bayesian Computation

Say we have a non-detection in a histogram bin (this could be interpreted as zero counts in a luminosity function bin). What is the most likely value of λ for the underlying Poisson distribution that generated this observation?

We can intuit, even without knowledge of Bayes' theorem, that the true value of λ might well not be 0, since it could be 0, 1 or even 10 with some chance of zero events being observed. We can see this in the plot below for different values of λ.

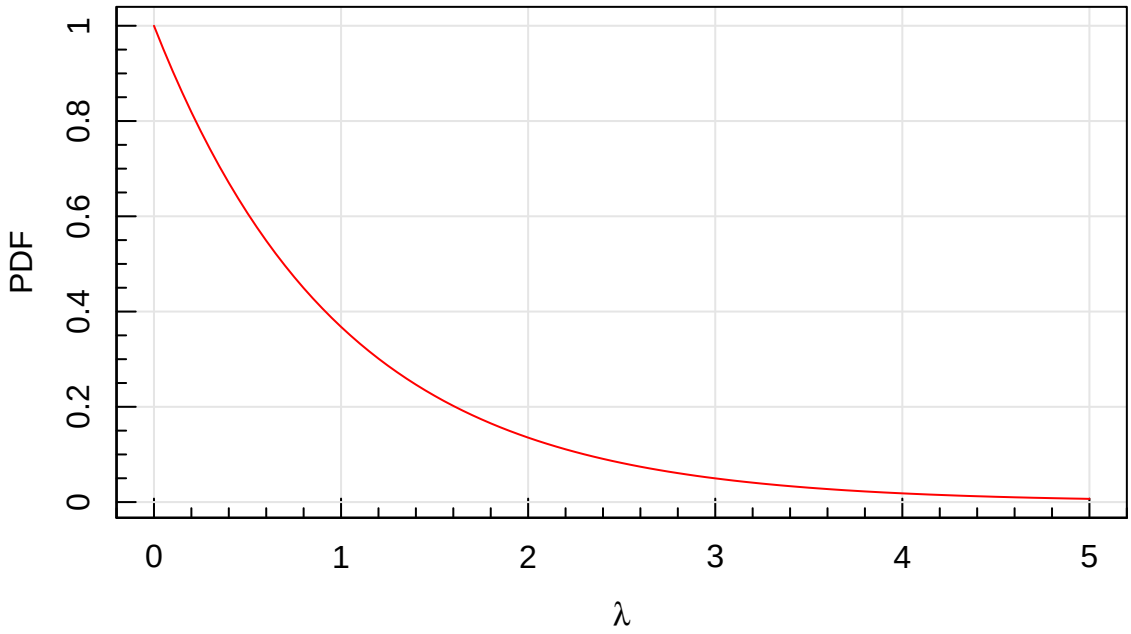
```
col_vec = hcl.colors(10)

magplot(0,0,type='n', xlab='x', ylab='PDF',xlim=c(0,5), ylim=c(0,1))
for(i in 1:10){
  lines(0:5,dpois(0:5,i-1),col=col_vec[i])
  points(0:5,dpois(0:5,i-1),col=col_vec[i])}
legend('topright',legend=paste('Lambda =',0:9),col=col_vec,lty=1,pch=1)
```



The 'prior' $p(\theta)$ is the probability of the model based on prior knowledge of how the input parameters might be distributed (in this case just λ). For the first approach at this problem our prior assumption is that there is an equal chance it could have *any* value (so Uniform). This will end up being a formally improper prior since it will not integrate to 1, but that will not affect the solution we are interest in here (more on this later). Arguably, we could apply a logarithmic Jeffreys Prior (see Danail's last part of the course), but for simplicity we will ignore that for now.

We now calculate the $p(y = 0, \theta)$ for *all* possible λ. In practice it will get very small very quickly, so we stop at $\lambda = 5$.

```
magcurve(dpois(x=0,lambda=x), from=0, to=5, type='l', col='red', xlab=expression(lambda),
         ylab='PDF')
```



The above is actually analytically a negative exponential distribution, so $p(\lambda) = e^{-\lambda}$. It is interesting to compute different averages on this posterior:

```
xval = seq(0,5,len=1000)
temp = dpois(0,xval)

xval[which.max(temp)] # Mode
```

```
## [1] 0
```

```
sum(temp*xval)/sum(temp) # Mean
```

```
## [1] 0.963718
```

```
tempsum = cumsum(temp)
xval[max(which(tempsum<=sum(temp)/2))] # Median
```

```
## [1] 0.6806807
```

Since we know the distribution is an exponential, we can calculate these numbers more accurately:

```
-log(0.5) #Median
```

```
## [1] 0.6931472
```

```
integrate(function(x){x*exp(-x)},0,Inf)$value / integrate(function(x){exp(-x)},0,Inf)$value #Mean
```

```
## [1] 1
```

We do not actually need to write the denominator out explicitly since it is already a true PDF (integrates to 1 under its domain), but we write it here for conceptual completeness.

Interpreting the Posterior

There are a few ways to think about the meaning of such a posterior PDF. The mode really is the "most likely" value (i.e. we would find it with a maximum likelihood scheme), but that is not always the most meaningful. Why is that? What would you do if I said you had to choose 1 or 2:6 on a die, where I would give you $1,000 if I roll a 1, and $1 otherwise? I hope you would pick 1, even though it is "less likely". i.e. the expectation really does matter in the real world.

If we observed 0 aid packages were required by a country in 1 year, the expectation of 1 per year is probably a more useful guide to planning ahead for 100 years worth of aid, i.e. we should buy 100 packages

for that period. For this reason the mean, or expectation, is often seen as the most useful measure. But it depends on the question being asked.

The above has the odd consequence that unless we have ultra-constraining priors the most likely results of witnessing 'nothing' is that there is 'something' (and potentially quite a lot of that thing). E.g. say I observe 0 bears in my office and there are 5 offices on my floor (setting an upper limit of 5 to the total possible bears given our somewhat small offices) what is the average number of bears per office?

```
integrate(function(x){x*exp(-x)},0,5)$value
```

```
## [1] 0.9595723
```

This would suggest I should never attempt to enter any office other than my own. So what to make of this? Well, in the real world we have exceedingly strong priors on almost all situations that negate naive Bayesian analysis. It is better to admit this and use an appropriate prior than to pretend we are being 'neutral' by not applying prior knowledge.

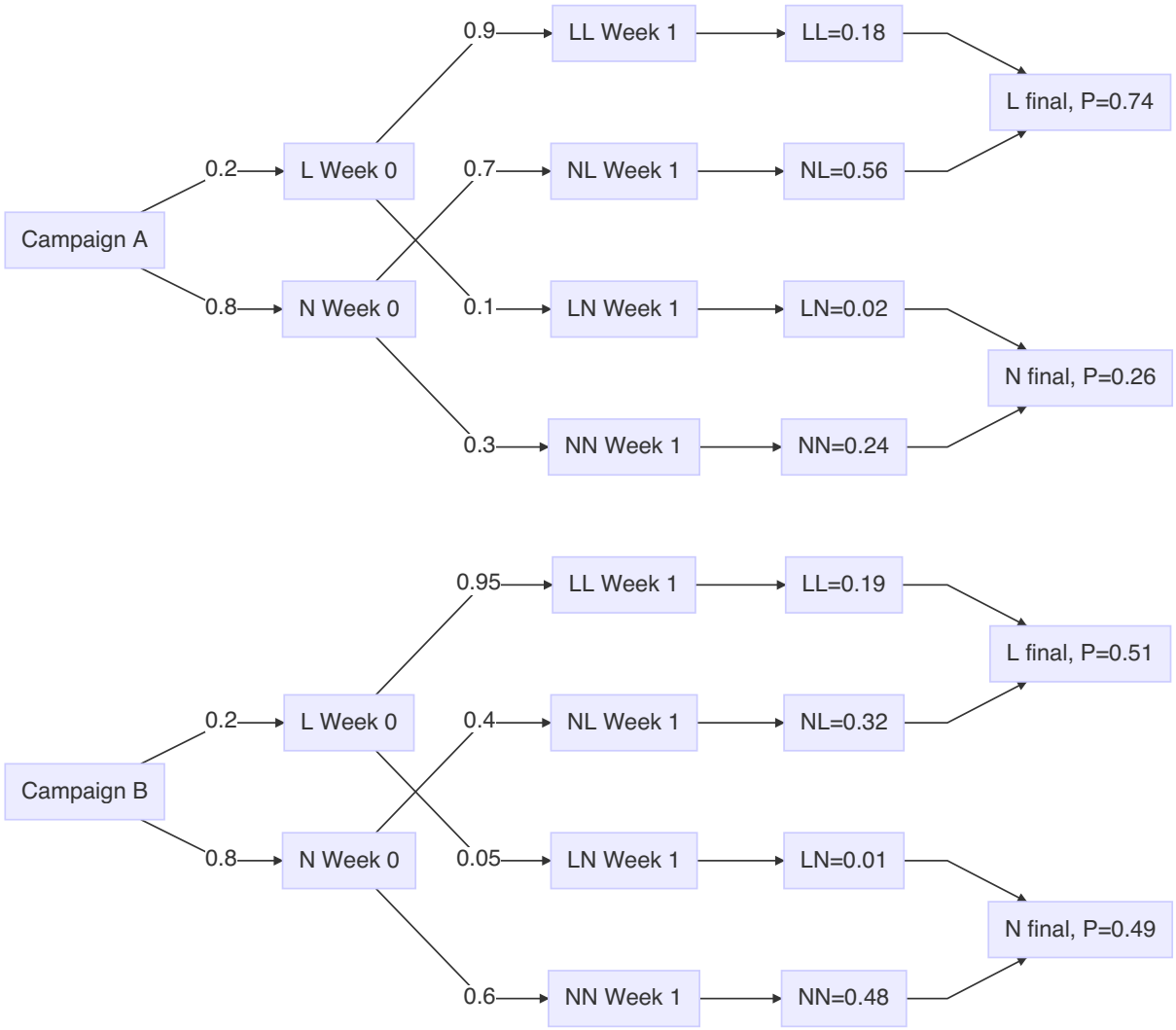
Building Probability Chains

Earlier we considered the Bayesian approach to determining disease tests, we will now look at a similar but more complicated problem. You are a band on *Spotify* and you would like more of your target demographic to listen to your music. There are two routes, those annoying voice ads that interrupt the music, or a more passive campaign that uses banner ads within the *Spotify* player.

Initially, 20% of your target demographic listens to your music.

- Per week, campaign A (voice ads) will convert 70% of non-listeners to listeners, but it will also turn off 10% of current listeners per week (because it is annoying!)
- Per week, campaign B (banner ads) will convert 40% of people of non-listeners to listeners, but will only turn off 5% of current listeners per week.

Which is the best campaign for one week? How about forever? We can visualise one week of campaigns easily via a probability tree (as seen earlier in the course):



The above tree gets quite complex for multiple iterations (e.g. we want to know the result after 20 weeks). However, we can take a very convenient short-cut by constructing a probability chain using matrix operations. This would look like:

$$A = \begin{bmatrix} 0.2 & 0.8 \end{bmatrix} \otimes \begin{bmatrix} 0.9 & 0.1 \\ 0.7 & 0.3 \end{bmatrix} = \begin{bmatrix} 0.74 & 0.26 \end{bmatrix} \quad (1)$$

$$B = \begin{bmatrix} 0.2 & 0.8 \end{bmatrix} \otimes \begin{bmatrix} 0.95 & 0.05 \\ 0.4 & 0.6 \end{bmatrix} = \begin{bmatrix} 0.51 & 0.49 \end{bmatrix} \quad (2)$$

This boils the above tree probabilities into a few simple matrix multiplication operations. We can do this in **R** easily:

```
domatmult = function(vec=matrix(c(0.2,0.8), ncol=2), mat, loop=21){
  vecall = {}
  for(i in 1:loop){
    vec = vec %*% mat
    vecall = rbind(vecall,vec)
  }
  return(list(final=vec, running=vecall))
}
```

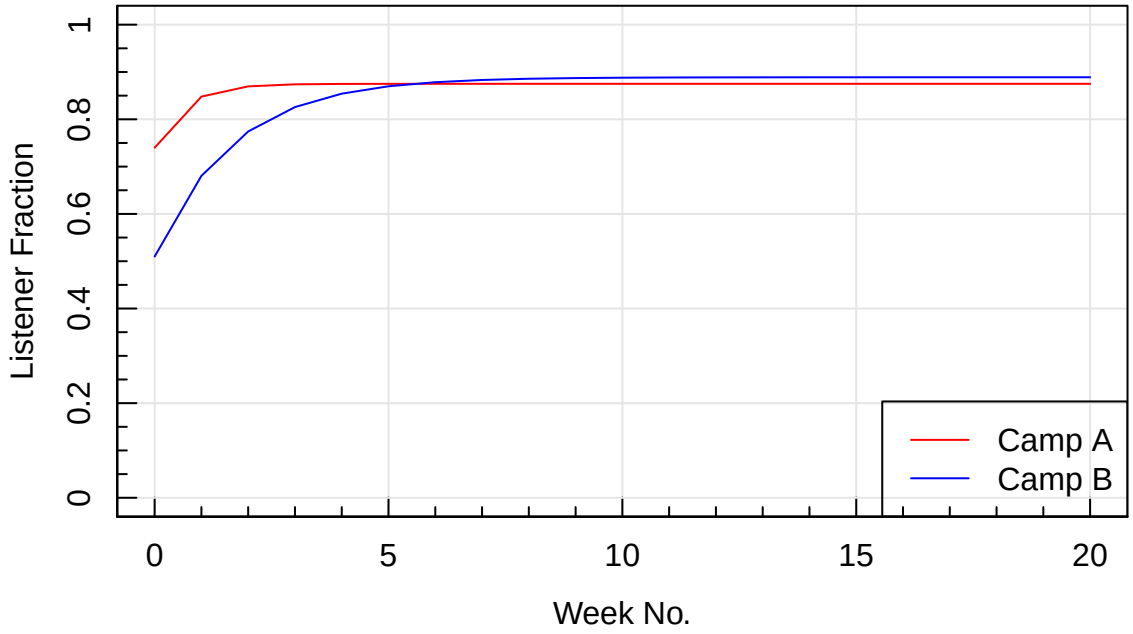
```
campA = domatmult(mat=matrix(c(0.9,0.7,0.1,0.3),ncol=2))
campA$final
```

```
##       [,1]  [,2]
## [1,] 0.875 0.125
```

```
campB = domatmult(mat=matrix(c(0.95,0.4,0.05,0.6),ncol=2))
campB$final
```

```
##           [,1]      [,2]
## [1,] 0.8888865 0.1111135
```

```
magplot(0:20, campA$running[,1], type='l', col='red', ylim=c(0,1), xlab='Week No.',
        ylab='Listener Fraction', grid=TRUE)
lines(0:20, campB$running[,1], col='blue')
legend('bottomright', legend=c('Camp A', 'Camp B'), col=c('red','blue'), lty=1)
```



So we can see that long-term it might pay not to have very annoying adverts, even if short term they seem to preferable (weeks 0–5).

Markov Chains

In the simplest sense we have just produced a 'Markov Chain' when computing our advert success convergence. What does this mean? As generally as possible, a Markov Chain is a mathematical system that undergoes transitions from one state (listeners=20%, non-listeners=80%) to another (listeners=74%, non-listeners=26%). Markov Chains are a means to calculate the end result (better known as the *posterior*) of complex systems numerically, where analytic results might be hard or impossible.

The key requirement to create a Markov chain is that each step can only depend on the preceding step, not the sequence of events that got us there (as we will see, there are ways around this requirement). This kind of memorylessness (it is a word!) is known as the 'Markov Property'. Famous examples are Brownian motion and the 'Drunkard's Walk'. Google's page-rank scheme uses (or at least it did when it was known how it worked) a Markov Chain analysis.

Simple Metropolis Hastings Algorithm

Let us look again at the 0 observations problem (with mode=0, mean=1, and median=0.69) from a bit earlier. To generically solve a Bayesian problem we need three things: data, a likelihood Model (usually a function that predicts the data), and a scheme that can produce the posterior distribution (we do not necessarily care how).

How can we explore the posterior space efficiently? In this case it is easy to do so directly since we do

not have much data and we have a limited range of plausible model space ($0 < \lambda < 10$). However we can conceptually do the following:

- If we choose a value x for λ, and compare that posterior likelihood to another value for $\lambda(x + dx)$, the ratio of likelihoods tells us the *relative* probabilities of the posterior PDF at those two locations.
- To walk through the posterior space we can do the following:
  - If the likelihood at $\lambda(x + dx)$ is higher than $\lambda(x)$ we update the posterior vector.
  - If the likelihood at $\lambda(x + dx)$ is lower than $\lambda(x)$ we accept it if a random number between 0 and the likelihood at $\lambda(x)$ is less than the likelihood at $\lambda(x + dx)$, so if the new likelihood is $2/3$ the previous likelihood we will update the posterior vector $2/3$ of the time.
- Doing this means we will slowly (noisily) march towards the best solution in our parameter space (we are more likely to move towards the correct answer than away from it).
- If we keep a chain of all posterior values, the PDF of this chain will reflect the posterior likelihood distribution. This is because the chance of appearing in this chain is directly related to the local *relative* likelihood of our posterior. The relative part is important, since it means our likelihood evaluations do not have to be *absolutely* correct, multiplying through by a random constant (i.e. not correctly normalising our likelihood) would achieve the same results.

Above we have just described the Metropolis Hastings (MH) Markov Chain Monte Carlo (MCMC) algorithm. An MCMC is a type of random stepping algorithm that obeys certain Markov Chain conditions, in particular it has to be memoryless, i.e. the next step can only be affected by its current state. Metropolis Hastings is a particularly simple type of MCMC.

If we remember simulated annealing from the a few lectures ago, there are actually many ways we can choose to move around our posterior space, i.e. there are many schemes to select dx. It could be a sample from a Uniform or Normal distribution, and it could even adapt with time. As long as we compare likelihoods between old and proposed positions we should be able to build up an image of the posterior PDF we care about.

We can write it in a very compact simple form below (ignoring explicit priors, i.e. we will have implicit improper Uniform priors):

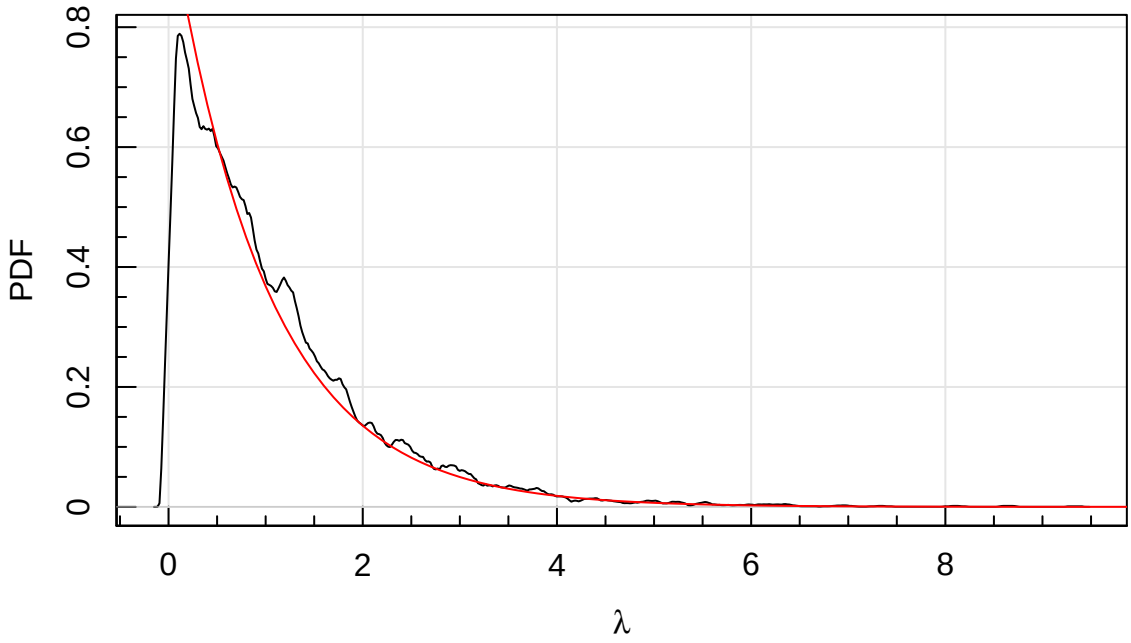
```
MetroHast_simp = function(Model, Data, parm=1, iterations=1e5, step=1){
  posterior = rep(NA,iterations); currentLL=Model(parm,Data)
  for(i in 1:iterations){
    trial = parm + rnorm(1,sd=step) #Here we choose dx from sampling the Normal
    trial = interval(trial, 0, 100) #This we stay between 0 and 100
    newLL = Model(trial,Data)
    if(newLL > currentLL){ #If new likelihood is higher keep it
        parm = trial; currentLL = newLL; posterior[i] = parm
    }else{ #If new likelihood is lower we test to see if we want to randomly update
        check = runif(1,0,exp(currentLL)) < exp(newLL)
        if(check){parm = trial; currentLL = newLL; posterior[i] = parm}
    }
  }
return(posterior[!is.na(posterior)])
}
```

Now we can run multiple iterations of this algorithm inputting our data (just a single observation of 0) and our likelihood model (not explicitly defining priors for now):

```
testMH1 = MetroHast_simp(Model=function(parm, Data){dpois(Data,parm,log=TRUE)}, Data=0, parm=1)
```

We can plot the density of the posterior we have sample and plot the analytic solution over the top.

```
magplot(density(testMH1[1:length(testMH1) %% 10==0], kern='rect', bw=0.05), xlab=expression(lambda),
curve(dpois(x=0,lambda=x), from=0, to=10, col='red', add=TRUE)
```

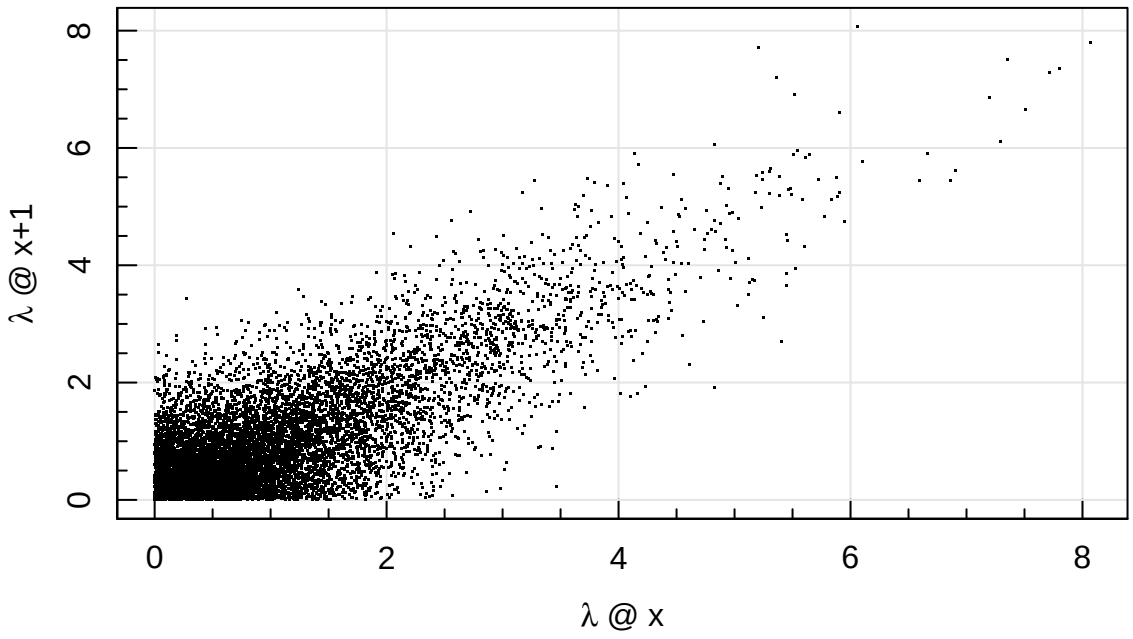


That looks pretty good! Take some time to go through the steps of how we did this, and convince yourself that the result above is what we should expect.

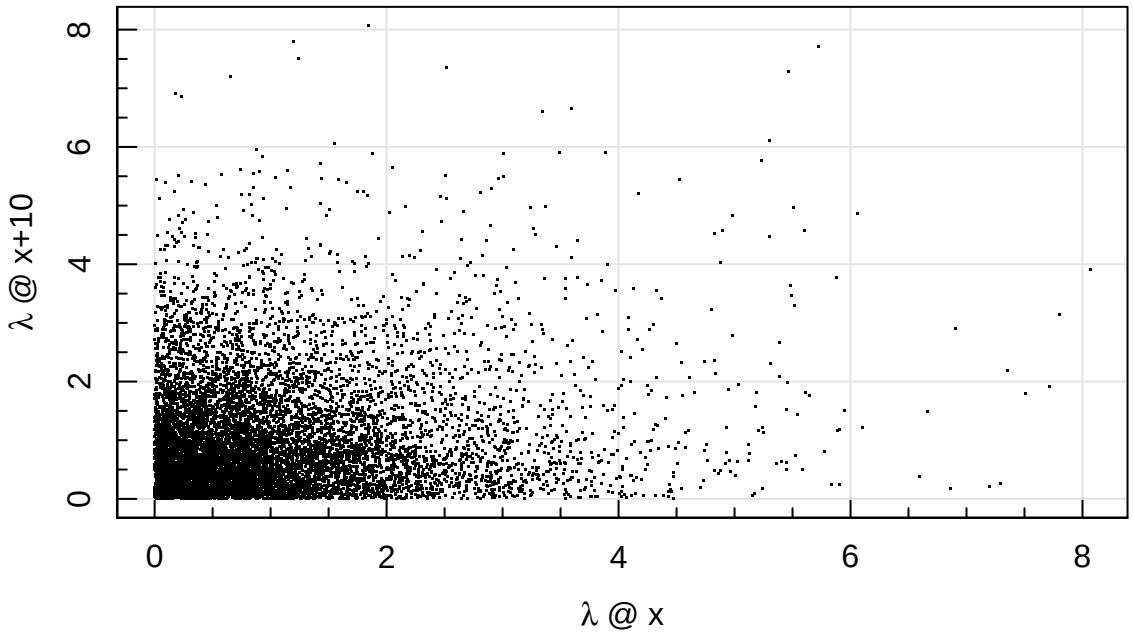
Thinning

You might have noticed above that we throw away 90% of our samples. This is to ensure the memoryless requirement of the Markov Chain. This process is known as thinning. How much we might want to thin the posterior depends on how auto-correlated the data is. We can see this by plotting the lagged values ('lagged' means we compare to outputs N steps away):

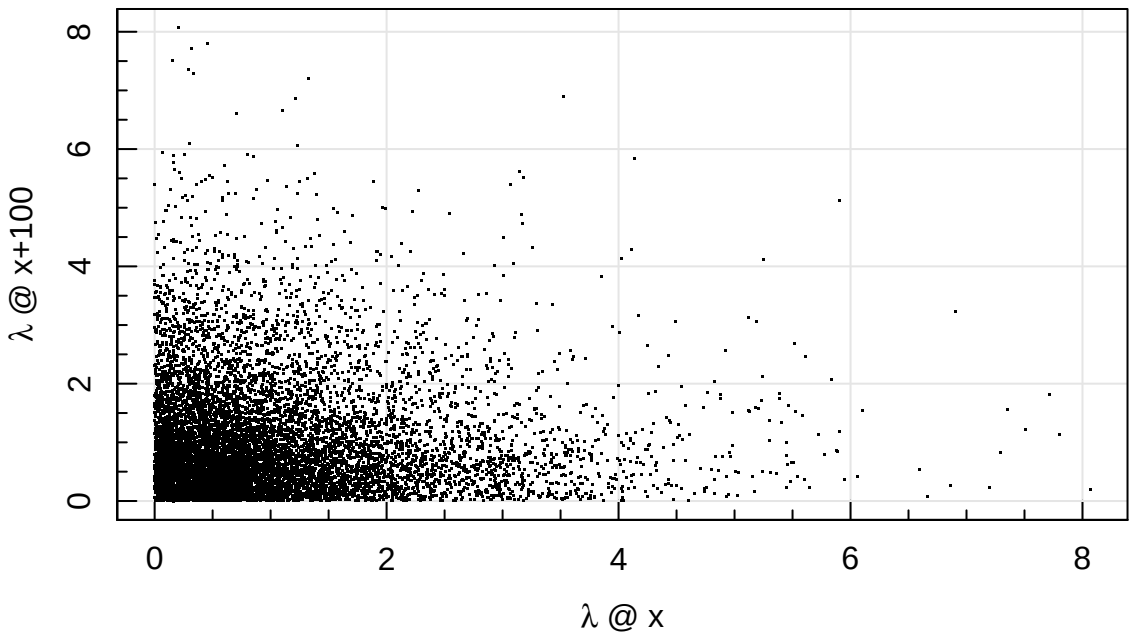
```
magplot(testMH1[1:1e4], testMH1[1:1e4+1], xlab=expression(lambda*' @ x'), ylab=expression(lambda*' @
```




```
magplot(testMH1[1:1e4], testMH1[1:1e4+10], xlab=expression(lambda*' @ x'), ylab=expression(lambda*'
```




```
magplot(testMH1[1:1e4], testMH1[1:1e4+100], xlab=expression(lambda*' @ x'), ylab=expression(lambda*'
```



Clearly we need some thinning, but there is not a huge improvement in the auto-correlation between a lag of 10 and 100, so we use the former to keep as many samples as possible (else we are just wasting computing effort for now reason).

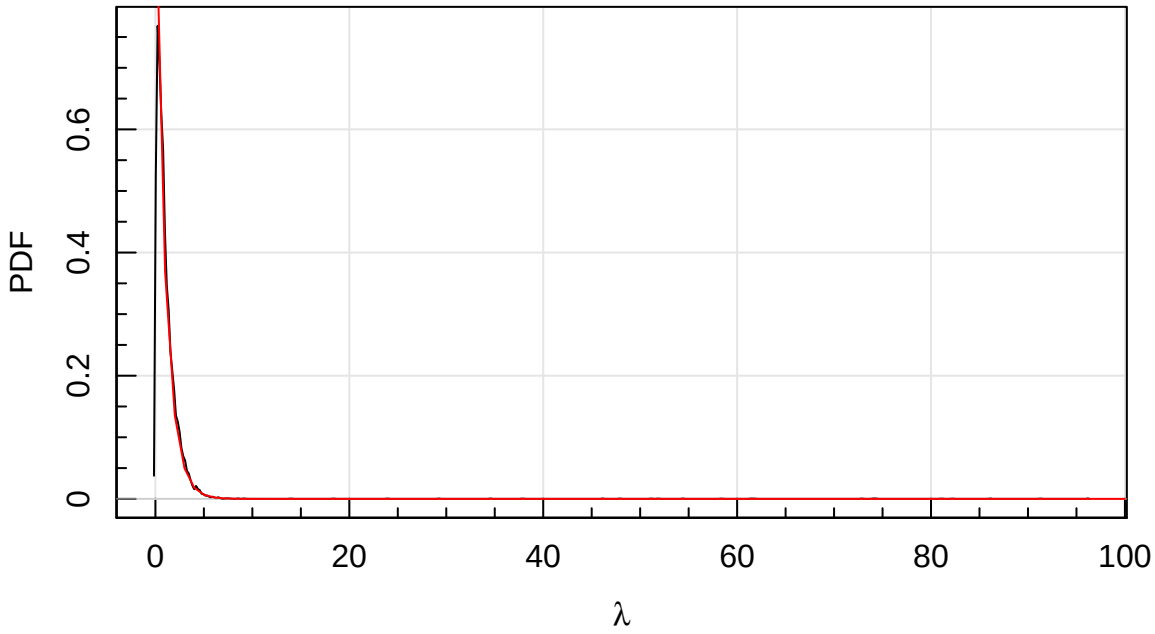
Burn In

In the example above we started pretty close to the expectation value (mean = 1), so we did not have to worry about finding maximising the LL to find the right value of λ. How would our MH algorithm behave if we started a long way from the correct solution?

```
testMH100 = MetroHast_simp(Model=function(parm, Data){dpois(Data,parm,log=TRUE)}, Data=0, parm=100)
```

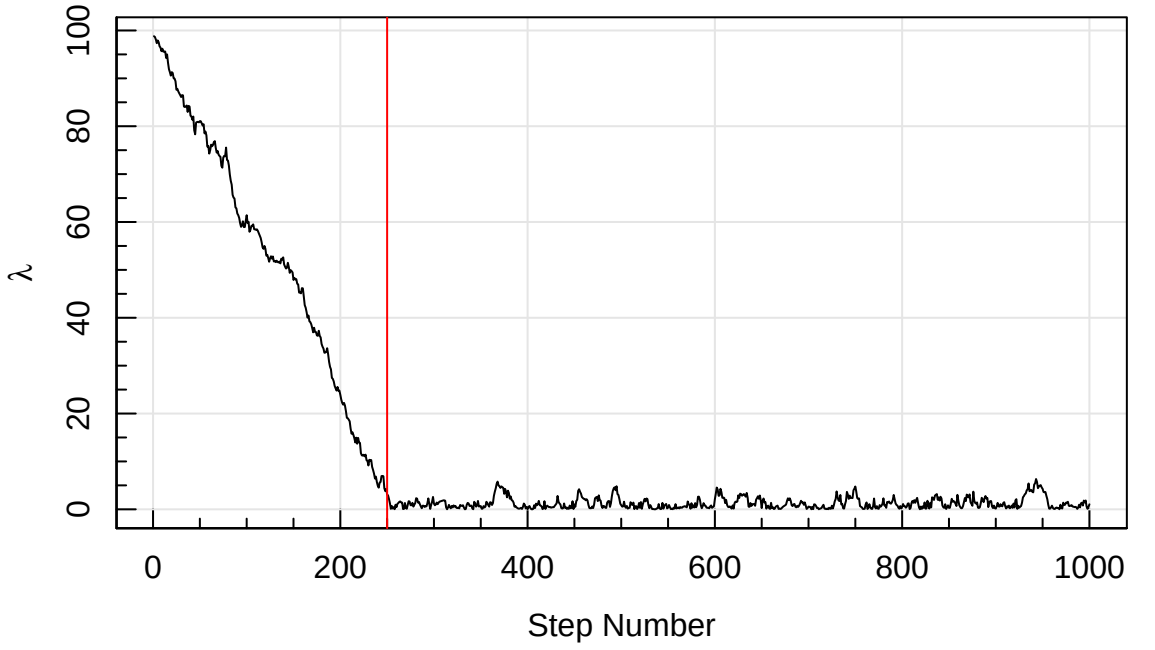
Plot the result:

```
magplot(density(testMH100[1:length(testMH100) %% 10==0], kern='rect', bw=0.05), xlab=expression(lamb
curve(dpois(x=0,lambda=x), from=0, to=100, col='red', add=TRUE)
```



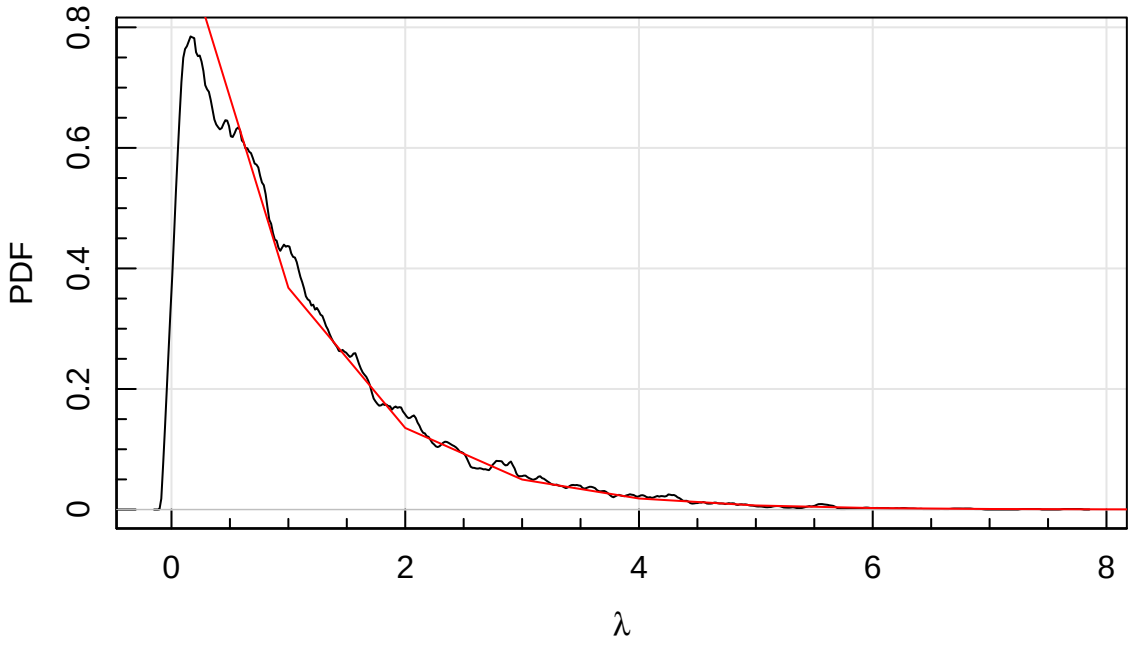
We can see in the above plot that there is a large tail of implausible λ being sampled. This is because by starting at $\lambda = 100$ and only allowing relatively small steps, it takes a long time to reach the reasonable region of the posterior.

```
magplot(testMH100[1:1e3], type='l', xlab='Step Number', ylab=expression(lambda))
abline(v=250, col='red')
```



So we can immediately see that the first 250 or so steps are spent just trying to get the posterior chain near to the right part of the posterior space, with the LL steadily dropping (bar some small backwards step jitters). This period is called the 'burn in', and is the time taken just to get near to interesting part of the posterior that we then want to explore to figure out its shape etc. This is usually just thrown away in later analysis, here we remove the first 250 posterior chain samples:

```
testMH100_burn = testMH100[250:length(testMH100)]
magplot(density(testMH100_burn[1:length(testMH100_burn) %% 10==0], kern='rect', bw=0.05), xlab=expre
curve(dpois(x=0,lambda=x), from=0, to=100, col='red', add=TRUE)
```



Adding Priors

The final major step to consider when building our first MH MCMC code is how to incorporate priors. From earlier in the course we see that these are multiplicative in linear space, so they just become additive terms in log probability space. The easiest way to incorporate this is to have a function that returns a logged-likelihood for the prior given a set of parameters, add this to the LL already calculated, and create a final logged posterior (LP) likelihood that we wish to explore with out MH MCMC. In our slightly modified MH algorithm, this function is called *Prior*:

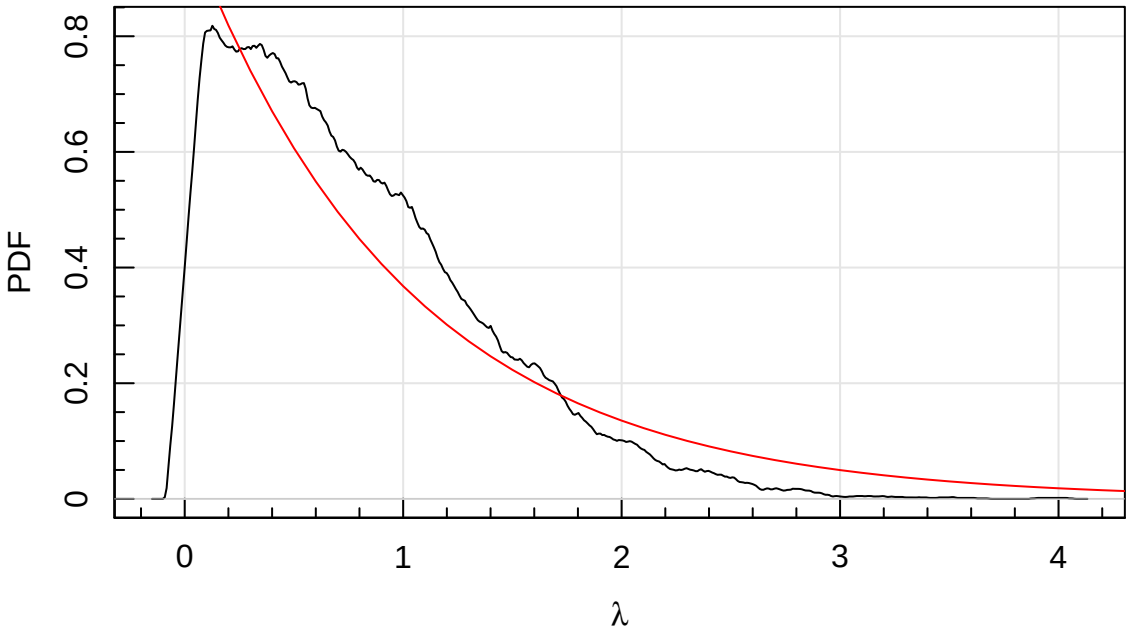
```
MetroHast_prior = function(Model, Data, Prior=function(parm){0}, parm=1, iterations=1e5, step=1){
  posterior = rep(NA,iterations); currentLP=Model(parm,Data)+Prior(parm)

  for(i in 1:iterations){
    trial = interval(parm+rnorm(1,sd=step), 0, 100) #Compact form of above code
    newLP = Model(trial,Data) + Prior(parm=trial) #The key new part, where we add a parameter prior
    if(newLP > currentLP){ #If new likelihood is higher keep it
        parm = trial; currentLP = newLP; posterior[i] = parm
    }else{ #If new likelihood is lower we test to see if we want to randomly update
        check = runif(1,0,exp(currentLP)) < exp(newLP)
        if(check){parm = trial; currentLP = newLP; posterior[i] = parm}
    }
  }
return(posterior[!is.na(posterior)])
}
```

We can run this, setting out prior model to be a Normal with $\mu = 1$ and $\sigma = 1$:

```
PriorFunc = function(parm){dnorm(x=parm, mean=1, sd=1, log=TRUE)}
testMH_prior = MetroHast_prior(Model=function(parm, Data){dpois(Data,parm,log=TRUE)},
                               Data=0, Prior=PriorFunc)
```

We can plot this again:

```
magplot(density(testMH_prior[1:length(testMH_prior) %% 10==0], kern='rect', bw=0.05), xlab=expressio
curve(dpois(x=0,lambda=x), from=0, to=10, col='red', add=TRUE)
```



Notice how the results are more concentrated around the value of 1 (as in there is a relative PDF excess) which is the centre of our Normal prior.

Conjugate Priors

Interestingly this actually looks worse now. The reason is the posterior actually has a PDF shape given by exponential distribution ($p(\lambda) = e^{-\lambda}$, check the maths of the Poisson when $x = 0$), and this is not *conjugate* with the Normal. This means it is not possible to produce an exponential posterior distribution with Normal priors. All distributions in the exponential family of distributions (most of the ones we have discussed belong to this class, except notably the Uniform and Pareto) have known conjugate prior distributions. In the case of the exponential distribution itself, the conjugate prior is given by the Gamma distribution.

Discussing this in detail is beyond the scope of this course, but you should remember the handy fact that the Normal is conjugate with itself, and in many real world cases we find the posterior to be Normal (or Multivariate-Normal), so it is appropriate to use Normal priors in many practical cases. It's one of the many reasons that the Normal approximation is pragmatically useful, and used so widely.

Marginal Likelihood

To start we will restate Bayes Theorem for a particular model M (we often make this explicit by adding a 'M' within our bracket notation):

$$p(\theta|x, M) = \frac{p(x|\theta, M)p(\theta, M)}{p(x, M)},$$

where $p(\theta|x, M)$ is our posterior PDF, $p(x|\theta, M)$ is the probability of the data given the model, $p(\theta, M)$ is our prior probability for given parameters for a particular model, and finally $p(x, M)$ is the prior predictive distribution. This is our normalising constant that gives the likelihood of the data for our model.

The marginal likelihood (ML) is simply the integrated likelihood or the prior predictive distribution of x in Bayesian inference. The log marginal likelihood (LML) is then just the natural log of this number. We can write this as:

$$p(x, M) = \int p(x|\theta, M)p(\theta, M)d\theta.$$

The prior predictive distribution indicates what x should look like, given the model, before x has been observed. I.e. $x_i \sim p(x_i|\theta, M)$ where θ is itself a sample from the prior distribution $\theta \sim p(\theta, M)$. The presence of the marginal likelihood of x normalises the joint posterior distribution ($p(\theta|x, M)$) ensuring it is a proper distribution (i.e. integrates to 1). Whilst it is simple to write, it is complex to compute- for a single realisation of parameters you in principle need to integrate over all possible models that could have generated them!

It is not always complex to compute though. If we consider our example of the single observation of 0 events from above, we can compute our prior free integral easily:

```
integrate(function(x){dpois(x=0,lambda=x)},lower=0,upper=Inf)
```

```
## 1 with absolute error < 5.7e-05
```

But we can add a prior to this to make it proper, else we are implicitly assuming an improper Uniform prior with a value of 1 over all domains:

```
integrate(function(x){dpois(x=0,lambda=x)*dunif(x=x, min=0, max=10)},lower=0,upper=10)
```

```
## 0.09999546 with absolute error < 2.8e-15
```

If we directly compare the ratio of $p(x, M)$ for different models we are computing what is known as the Bayes Factor, where larger values indicates a preference for one model over another (in the regime where our models and priors are well specified anyway).

We can try a few different priors:

```
integrate(function(x){dpois(x=0,lambda=x)*dunif(x=x, min=0, max=2)},lower=0,upper=Inf)
```

```
## 0.4323324 with absolute error < 4e-05
```

```
integrate(function(x){dpois(x=0,lambda=x)*dunif(x=x, min=1, max=10)},lower=0,upper=Inf)
```

```
## 0.04087363 with absolute error < 7e-05
```

```
integrate(function(x){dpois(x=0,lambda=x)*dunif(x=x, min=0, max=100)},lower=0,upper=Inf)
```

```
## 0.009999998 with absolute error < 3.5e-05
```

```
integrate(function(x){dpois(x=0,lambda=x)*dnorm(x=x,mean=1,sd=1)},lower=0,upper=Inf)
```

```
## 0.3032653 with absolute error < 2.8e-05
```

```
integrate(function(x){dpois(x=0,lambda=x)*dnorm(x=x,mean=1,sd=10)},lower=0,upper=Inf)
```

```
## 0.03969904 with absolute error < 7.2e-07
```

Oh dear! Clearly our marginal likelihood (and therefore our Bayes Factor for ratios between models) is highly sensitive to our choice of priors, much more so that the shape of the posterior distribution that informs us about our model parameters (θ). Perhaps most alarmingly merely extending out Uniform distribution far beyond the domain of the true model (remember, the posterior converges to 0 by $\lambda = 5$) hugely reduces the ML. It is why it is often said that Bayesian analysis is not the best choice for model selection. It works best in the domain when we *know* our true model M, and we merely want to explore the posterior of the parameters that define it given our data.

The above process would work if we actually chose appropriate priors given our model data combination, i.e. it is quite obvious the parameters should not have Uniform priors over a large support domain. This has been discussed previously with reference to Jeffreys priors, but computing such 'proper priors' is rarely undertaken in real world problems because it is often considered too computationally expensive.

In summary, proper marginal likelihoods are usually difficult to compute in all but the simplest cases, but using our posterior samples from running an MCMC we can estimate it using a number of approaches. These will not be discussed in detail here since in practice we tend to use more practical, tractable and computationally stable methods for model selection (discussed towards the end of this chapter).

Advanced Metropolis Hastings

We will now construct our fairly fully-featured MH algorithm, with the addition of parameters that control the burn in discarding (what fraction of beginning samples are ignored as we search towards the global maximum in LL) and the thinning (where $1/thin$ is the fraction of samples kept).

```
MetroHast_adv = function(Model, Data, Prior=function(parm){0}, parm=1, iterations=1e5,
                         step=1, burn=0.1, thin=10){
  Nparm = length(parm)
  keep = vector(length=iterations)
  posterior = matrix(NA,iterations,Nparm); colnames(posterior)=names(parm)
  outLL = vector(length=iterations)
  outLP = vector(length=iterations)
  currentLL = Model(parm,Data)
  currentPrior = Prior(parm)
  currentLP = currentLL+currentPrior
  for(i in 1:iterations){
    if((i %% (iterations/10))==0){print(i)}
    trial = parm + rnorm(Nparm, sd=step)
    newLL = Model(trial,Data)
    newLP = newLL + Prior(parm=trial)
    if(newLP > currentLP){ #If new likelihood is higher keep it
      parm = trial
      currentLP = newLP
      currentLL = newLL
      keep[i] = TRUE
    }else{ #If new likelihood is lower we test to see if we want to randomly update
      check = runif(1,0,exp(currentLP)) < exp(newLP)
      if(check){
        parm = trial; currentLP = newLP ;currentLL = newLL; keep[i] = TRUE
      }else{
        keep[i] = FALSE
      }
    }
    posterior[i,] = parm
    outLL[i] = currentLL
    outLP[i] = currentLP
  }
  #Remove the burn in
  burnignore = 1:iterations > burn*iterations
  posterior = as.matrix(posterior[burnignore,])
  outLL = outLL[burnignore]
  outLP = outLP[burnignore]
  keep = keep[burnignore]
  if(thin > 1){
    #Thin out our samples
    thinlogic = 1:length(outLL) %% thin ==0
    posterior = as.matrix(posterior[thinlogic,])
    outLL = outLL[thinlogic]
    outLP = outLP[thinlogic]
  }

  output = list(posterior=posterior, acc_rate=length(which(keep))/((1-burn)*iterations),
                LP=outLP, LL=outLL, Nparm=Nparm, Nsamp=length(outLP))
  class(output)='MetroHast' #Notice we set the object class to MetroHast
  return(output)
}
```

We now write a short function that will take our posterior samples and plot the chains. This is known as

a 'triangle plot', or 'corner plot'.

```
plot.MetroHast = function(x){ #This means we can run 'plot' on objects of class MetroHast
  if(x$Nparm == 1){
    lineloc = quantile(x$posterior,pnorm(c(-1,0,1)))
    magplot(density(x$posterior))
    points(x$posterior,rep(0,x$Nsamp), pch='|', col=hsv(v=0,alpha=0.1), cex=0.5)
    abline(v=lineloc[2], lty=2, col='red')
    abline(v=lineloc[c(1,3)], lty=3, col='red')
  }else{
    suppressWarnings({magtri(x$posterior)})
  }
}
```

A More Complex Example

Now we have a fairly fully featured MH function, we will try a more complex problem. Here we will generate data from a Normal distribution, and then try to fit for μ and σ, making use of priors, thinning and burn-in. First we make our key functions:

```
MH_Model = function(parm, Data){sum(dnorm(Data, mean=parm[1], sd=10^parm[2], log=TRUE))}
MH_Data = rnorm(1e2, mean=1, sd=1)
MH_Prior = function(parm){dnorm(parm[1], 1, 1, log=TRUE) + dnorm(parm[2], 0, 1, log=TRUE)}
```

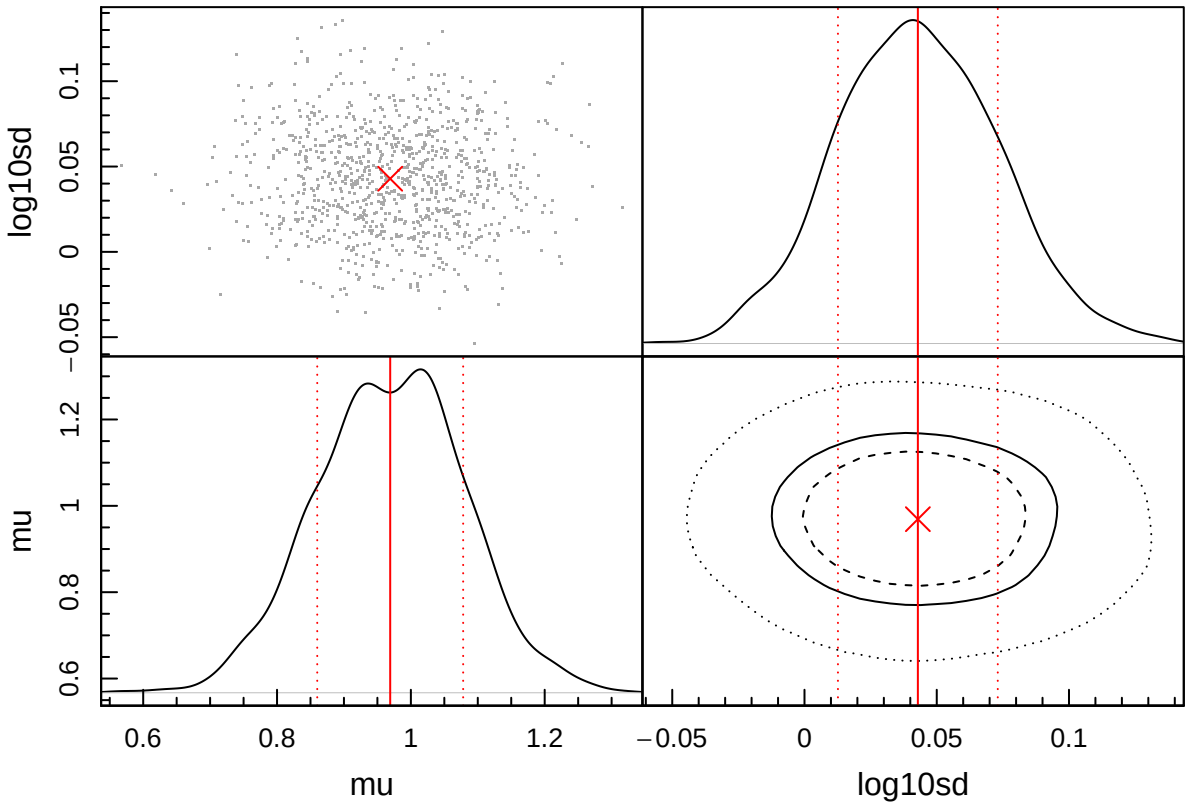
Then we fit it using our advanced MH function:

```
MCMC = MetroHast_adv(Model=MH_Model, Data=MH_Data, Prior=MH_Prior, parm=c(mu=1, log10sd=0), step=0.1
```

```
## [1] 10000
## [1] 20000
## [1] 30000
## [1] 40000
## [1] 50000
## [1] 60000
## [1] 70000
## [1] 80000
## [1] 90000
## [1] 100000
```

Since we made a specialised class function **plot.MetroHast** we can now easily produce the triangle plots that shows the posteriors of our model:

```
plot(MCMC)
```



We know that in the input model had $\mu = 1$ and $\log_{10}\sigma = 0$, so the results look pretty convincing and we see the posterior samples suggest the true model exists within the 50% significance contours.

Using Laplace's Demon

Rather than use our own sampler, **R** gives us access to a large number via the easy to use **LaplacesDemon** (*LD*) package. This has all of the functionality discussed so far, and a lot more on top. We can load it in the usual way (if not already loaded):

```
library(LaplacesDemon)
```

One downside is we have to construct our model and data in a very particular form. The data we used before (one observation of 0 events) now looks like:

```
Data_0 = list(data=0, mon.names='', parm.names='lambda', N=1)
```

Where *data* is the data, *mon.names* are the monitored variable names (should be same length as the *Monitor* vector below), *parm.names* is a character vector of parameter names and must be the same length as the number of parameters being fit, and N is the length of the number of observations.

And the model has to return a few more things (not just the LL or LP):

```
Model_pois = function(parm, Data){
  parm = interval(parm, 0, 100)
  val.prior = 0
  LL = sum(dpois(Data$data, lambda=parm, log=TRUE))
  LP = LL + val.prior
  return(list(LP=LP, Dev=-2*LL, Monitor=1, yhat=1, parm=parm))
}
```

Where LP is the log-posterior likelihood (log-likelihood plus log-prior likelihood), *Dev* is the deviance, which is χ^2 ignoring the offset term, *Monitor* is any additional outputs that we wish to track (set to a dummy value of 1 here), *yhat* is an optional posterior check (set to a dummy value of 1 here) and *parm* are the current parameters being assessed.

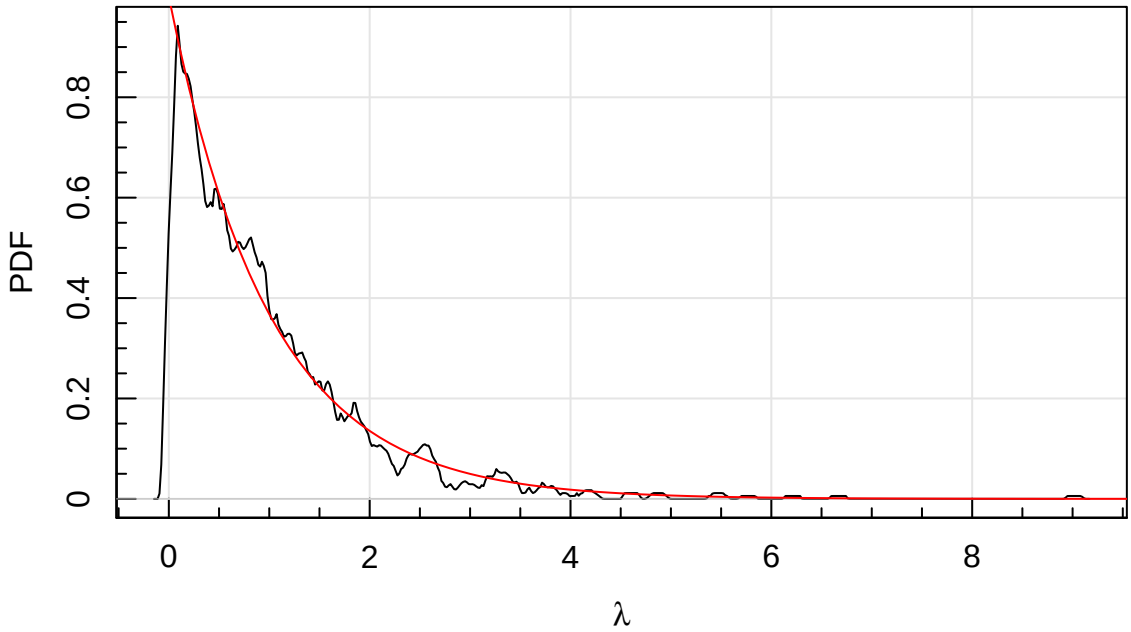
And now we can run it with one of the nicer MCMC samplers (often a good default option) component-wise hit-and-run metropolis (CHARM). This has the nice features of local exploration (like MH), but is able to take big steps to get out of local maxima in LP space (hit-and-run), and runs on parameters one at a time (component-wise) so can overcome serious covariance in the parameter posteriors.

```
output_0 = LaplacesDemon(Model_pois, Data_0, Initial.Values=1, Algorithm='CHARM',
                         Iterations=1e4, Thinning=10, Status=Inf)
```

```
##
## Laplace's Demon was called on Mon Sep 15 15:51:01 2025
##
## Performing initial checks...
## 'Status' has been changed to 10000.
## Algorithm: Componentwise Hit-And-Run Metropolis
##
## Laplace's Demon is beginning to update...
## Iteration: 10000,   Proposal: Componentwise,   LP: -0.2
##
## Assessing Stationarity
## Assessing Thinning and ESS
## Creating Summaries
## Estimating Log of the Marginal Likelihood
## Creating Output
##
## Laplace's Demon has finished.
```

And we again plot the result:

```
magplot(density(output_0$Posterior2, kern='rect', bw=0.05), xlab=expression(lambda), ylab='PDF')
curve(dpois(x=0,lambda=x), from=0, to=10, col='red', add=TRUE)
```



Above we plot *posterior2*. This is because *LD* makes an assessment of the posterior chains and only keeps the well converged ones in *posterior2* (*posterior1* contains all of them), so this naturally removes the burn in phase of the MCMC.

Handily **LD** also computes the log marginal likelihood (LML) for us, using some fairly sophisticated approaches depending on the posterior properties.

```
exp(output_0$LML)
```

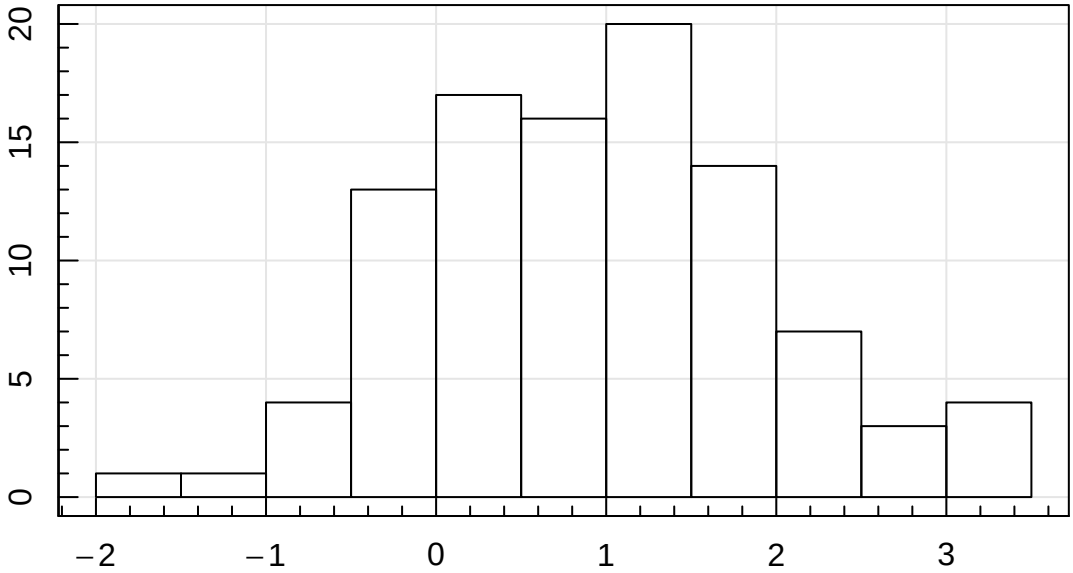
```
## [1] 0.9882989
```

This is very close 1, which is what we are expecting/hoping.

Impact of Priors

With this simple example we will now explore the effect of making a poor choice re our priors. First we will sample 100 Normal random numbers with $\mu = 1$ and $\sigma = 1$.

```
set.seed(666)
random_samples = rnorm(1e2, mean=1, sd=1)
Data1_100 = list(data=random_samples, mon.names='', parm.names='mean', N=1e2)
Data2_100 = list(data=random_samples, mon.names='', parm.names=c('mean','sd'), N=1e2)
maghist(random_samples, verbose = FALSE)
```



Now we make 4 different models to test, with different priors and number of parameters.

The single parameter model is constructed such that we only infer μ, σ is already set to the correct value (1).

```
Model1a = function(parm, Data){
  val.prior = dnorm(parm, 1, 1, log=TRUE) #prior sd=1
  LL = sum(dnorm(Data$data ,mean=parm, sd=1, log=TRUE))
  LP = LL + val.prior
  Modelout = list(LP=LP, Dev=-2*LL, Monitor=1, yhat=1, parm=parm)
  return(Modelout)
}
```

```
Model1b = function(parm, Data){
  val.prior = dnorm(parm, 1, 10, log=TRUE) #prior sd=10
  LL = sum(dnorm(Data$data, mean=parm, sd=1, log=TRUE))
  LP = LL + val.prior
  Modelout = list(LP=LP, Dev=-2*LL, Monitor=1, yhat=1, parm=parm)
  return(Modelout)
}
```

The 2 parameter model allows us to infer μ and σ.

```
Model2a = function(parm, Data){
  val.prior = dnorm(parm[1], 1, 1, log=TRUE) + dnorm(parm[2], 0, 1, log=TRUE)  #prior sd=1
  LL = sum(dnorm(Data$data, mean=parm[1], sd=10^parm[2], log=TRUE))
  LP = LL + val.prior
  Modelout = list(LP=LP, Dev=-2*LL, Monitor=1, yhat=1, parm=parm)
```

```
  return(Modelout)
}
```

```
Model2b = function(parm, Data){
  val.prior = dnorm(parm[1], 1, 10, log=TRUE) + dnorm(parm[2], 0, 10, log=TRUE)  #prior sd=10
  LL = sum(dnorm(Data$data, mean=parm[1], sd=10^parm[2], log=TRUE))
  LP = LL + val.prior
  Modelout = list(LP=LP, Dev=-2*LL, Monitor=1, yhat=1, parm=parm)
  return(Modelout)
}
```

We can sample the 4 different model options using **LaplacesDemon**:

```
output1a_100 = LaplacesDemon(Model1a, Data1_100, Initial.Values=1, Algorithm='CHARM')
output1b_100 = LaplacesDemon(Model1b, Data1_100, Initial.Values=1, Algorithm='CHARM')
output2a_100 = LaplacesDemon(Model2a, Data2_100, Initial.Values=c(1,0), Algorithm='CHARM')
output2b_100 = LaplacesDemon(Model2b, Data2_100, Initial.Values=c(1,0), Algorithm='CHARM')
```

We now create a function to calculate Bayes Factors using the LML that **LD** computes internally:

```
BFcalc = function(LMLlist){
  N_LML = length(LMLlist)
  for(i in 1:N_LML){
    if(class(LMLlist[[i]])=='demonoid'){
      LMLlist[[i]]=LMLlist[[i]]$LML
    }
  }
  M_BF = matrix(0,N_LML,N_LML)
  post.prob = vector(length=N_LML)
  for(i in 1:N_LML){
    post.prob[i] = exp(LMLlist[[i]])
    for(j in 1:N_LML){
      M_BF[i,j] = exp(LMLlist[[i]] - LMLlist[[j]])
    }
  }
  post.prob = post.prob/sum(post.prob)
  return(list(M_BF=M_BF, post.prob=post.prob))
}
```

We can then create a grid of Bayes Factors to select a preferred model.

```
BFcalc(list(output1a_100, output1b_100, output2a_100, output2b_100))
```

```
## $M_BF
##             [,1]       [,2]       [,3]      [,4]
## [1,] 1.00000000 0.60650199 12.1903758 12.294351
## [2,] 1.64879920 1.00000000 20.0994819 20.270916
## [3,] 0.08203193 0.04975253  1.0000000  1.008529
## [4,] 0.08133817 0.04933176  0.9915429  1.000000
##
## $post.prob
## [1] 0.35559737 0.58630866 0.02917034 0.02892364
```

As should be expected, we prefer our single parameter models (large Bayes Factors in the top-right of the grid) that have the intrinsic σ in them already. Of the two single parameter models, there is "barely worth mentioning" " support for Model1b with the looser prior distribution around μ and σ (10 versus 1). I.e. our choice in priors, although very different, actually has very little impact on which model we choose. This is good, because we want model selection to be largely driven by the data not the priors.

But what if we make the assumed distribution σ different to the true value: 1.2 rather that 1. Will we prefer the two parameter model then?

```
Model1a_bad = function(parm, Data){
val.prior = dnorm(parm, 1, 1, log=TRUE) #prior sd=1
LL = sum(dnorm(Data$data ,mean=parm, sd=1.2, log=TRUE)) #sd=1.2
LP = LL + val.prior
Modelout = list(LP=LP, Dev=-2*LL, Monitor=1, yhat=1, parm=parm)}
```

```
Model1b_bad = function(parm, Data){
val.prior = dnorm(parm, 1, 10, log=TRUE) #prior sd=10
LL = sum(dnorm(Data$data, mean=parm, sd=1.2, log=TRUE)) #sd=1.2
LP = LL + val.prior
Modelout = list(LP=LP, Dev=-2*LL, Monitor=1, yhat=1, parm=parm)}
```

Make some more samples with **LaplacesDemon**.

```
output1a_100_bad = LaplacesDemon(Model1a_bad, Data1_100, Initial.Values=1, Algorithm='CHARM')
output1b_100_bad = LaplacesDemon(Model1b_bad, Data1_100, Initial.Values=1, Algorithm='CHARM')
```

Create a new grid of Bayes Factors to select a preferred model.

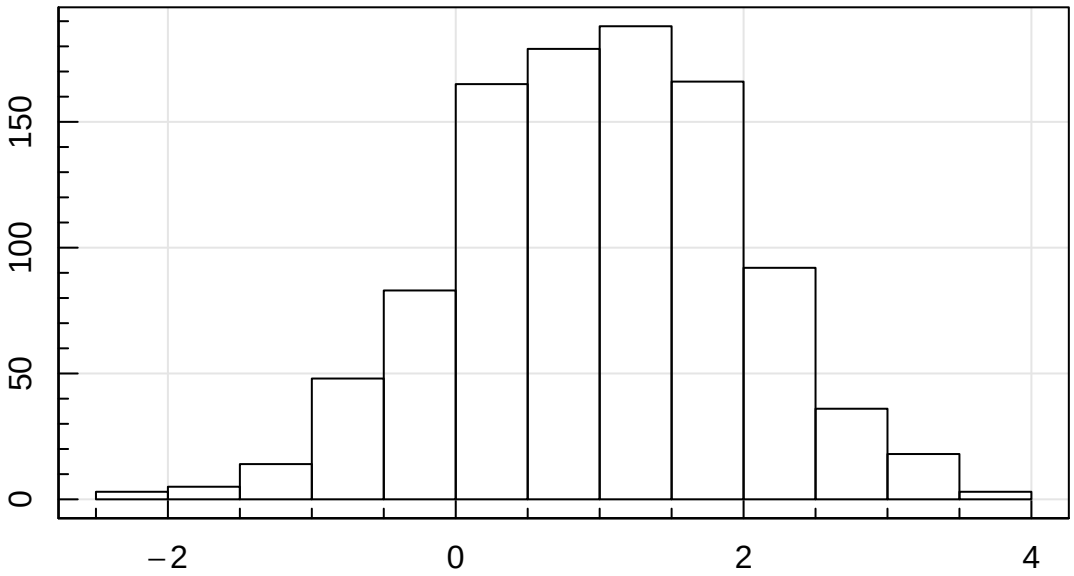
```
BFcalc(list(output1a_100_bad, output1b_100_bad, output2a_100, output2b_100))
```

```
## $M_BF
##           [,1]      [,2]      [,3]     [,4]
## [1,] 1.0000000 1.0018361 1.5764667 1.589913
## [2,] 0.9981673 1.0000000 1.5735775 1.586999
## [3,] 0.6343299 0.6354946 1.0000000 1.008529
## [4,] 0.6289653 0.6301201 0.9915429 1.000000
##
## $post.prob
## [1] 0.3066109 0.3060490 0.1944925 0.1928476
```

We still marginally prefer the simpler (though a bit incorrect) models. Not by as much though, so they have lost some support.

But what if we have more data? We will now create data with 1000 observations.

```
set.seed(666)
random_samples = rnorm(1000, mean=1, sd=1)
Data1_1000 = list(data=random_samples, mon.names='', parm.names='mean', N=1e3)
Data2_1000 = list(data=random_samples, mon.names='', parm.names=c('mean','sd'), N=1e3)
maghist(random_samples, verbose=FALSE)
```



Make some more samples with **LaplacesDemon**.

```
output1a_1000_bad = LaplacesDemon(Model1a_bad, Data1_1000, Initial.Values=1, Algorithm='CHARM')
output1b_1000_bad = LaplacesDemon(Model1b_bad, Data1_1000, Initial.Values=1, Algorithm='CHARM')
output2a_1000 = LaplacesDemon(Model2a, Data2_1000, Initial.Values=c(1,0), Algorithm='CHARM')
output2b_1000 = LaplacesDemon(Model2b, Data2_1000, Initial.Values=c(1,0), Algorithm='CHARM')
```

Create a new grid of Bayes Factors to select a preferred model.

```
BFcalc(list(output1a_1000_bad, output1b_1000_bad, output2a_1000, output2b_1000))
```

```
## $M_BF
##              [,1]         [,2]         [,3]         [,4]
## [1,] 1.000000e+00 5.190648e-01 2.512084e-14 3.131247e-14
## [2,] 1.926542e+00 1.000000e+00 4.839634e-14 6.032477e-14
## [3,] 3.980759e+13 2.066272e+13 1.000000e+00 1.246474e+00
## [4,] 3.193616e+13 1.657694e+13 8.022631e-01 1.000000e+00
##
## $post.prob
## [1] NaN NaN NaN NaN
```

With 10 times more data we can strongly reject both of the deliberately bad, but simpler, models. The two more complex models with different priors are almost indistinguishable in terms of selection preference.

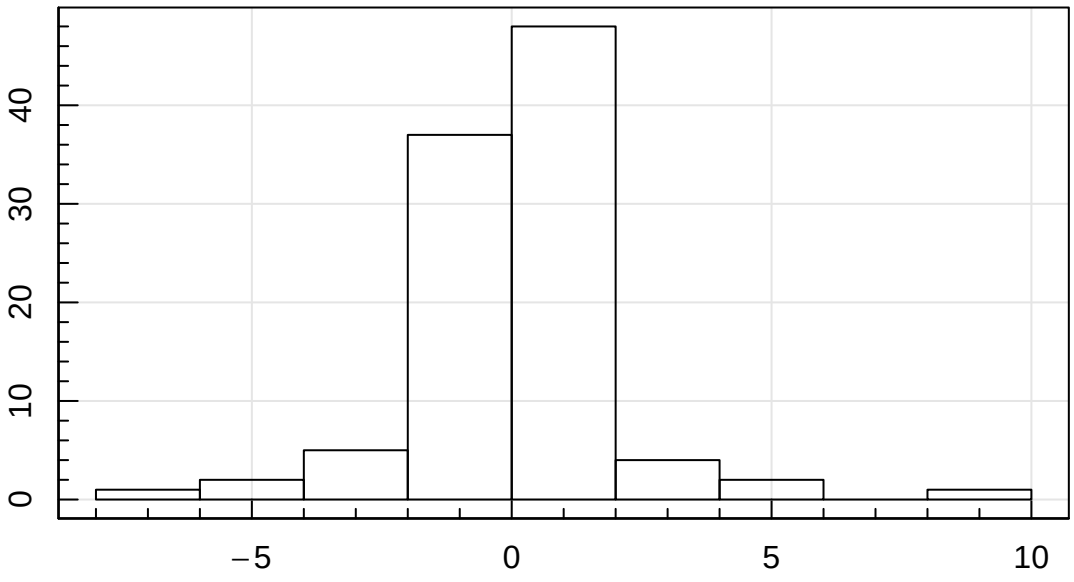
The conclusion to this is that our choice of priors has very little impact on our model selection. This means you should not worry too much about the precise prior ranges, as long as they are either correctly informative and conjugate when restrictive, or properly normalised when uninformative.

The Wrong Model

A common situation in the real world is that you are trying to fit slightly the wrong model to the data (we are not psychic after all).

Imagine for instance we generate Student-t distributed data and then try to fit it with either a Normal or Student-t:

```
set.seed(666)
random_samples = rt(100, df=3)
maghist(random_samples, verbose=FALSE)
```



We set up our data objects and model functions as before, with no explicit prior:

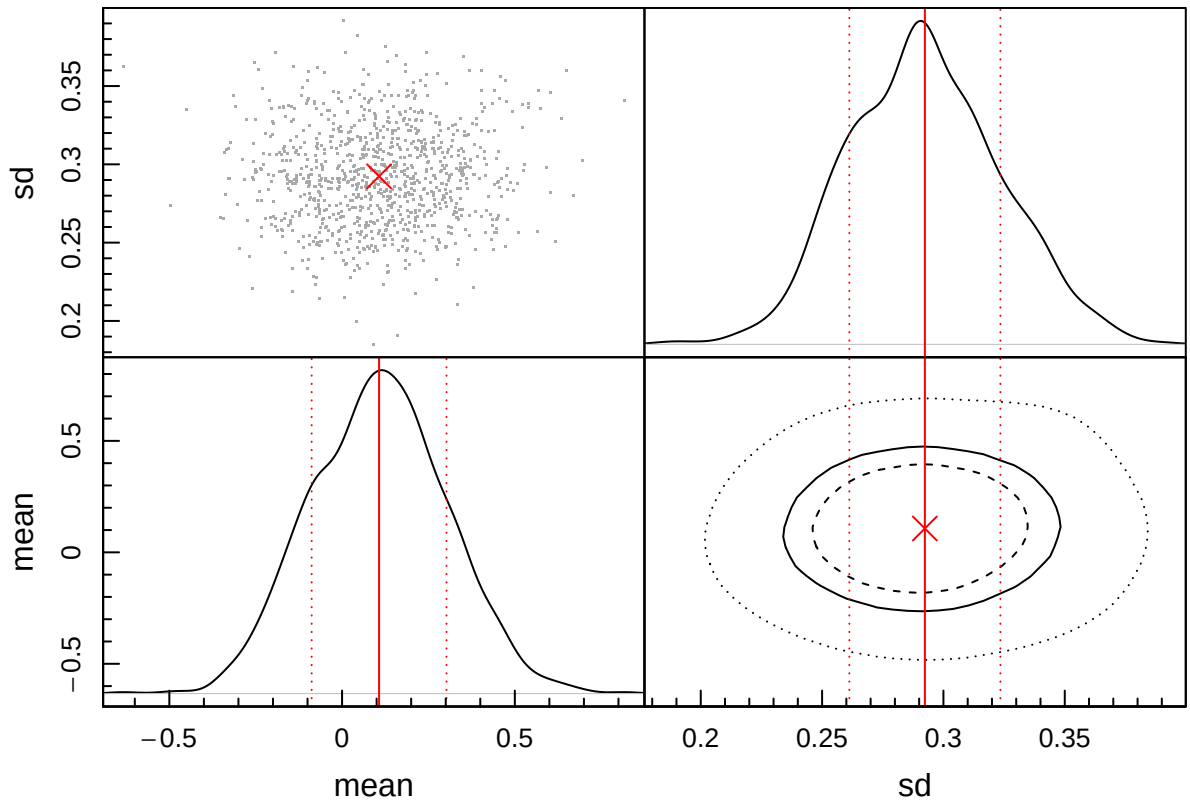
```
Data_ST1a_100 = list(data=random_samples, mon.names='', parm.names=c('mean','sd'), N=1e2)

Model1_ST = function(parm, Data){
  val.prior = 0
  LL = sum(dnorm(Data$data ,mean=parm[1], sd=10^parm[2], log=TRUE))
  LP = LL + val.prior
  Modelout = list(LP=LP, Dev=-2*LL, Monitor=1, yhat=1, parm=parm)
  return(Modelout)
}
```

```
Data_ST2a_100 = list(data=random_samples, mon.names='', parm.names='df', N=1e2)

Model2_ST = function(parm, Data){
  val.prior = 0
  LL = sum(dt(Data$data, df=10^parm, log=TRUE))
  LP = LL + val.prior
  Modelout = list(LP=LP, Dev=-2*LL, Monitor=1, yhat=1, parm=parm)
  return(Modelout)
}
```

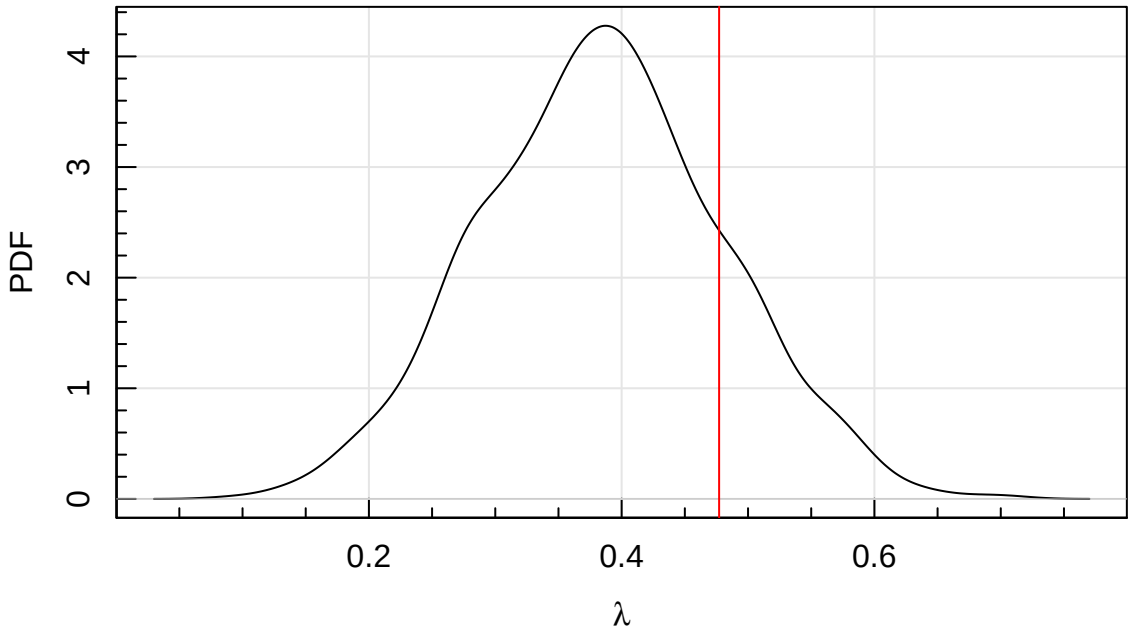
And run LD:

```
MCMC_ST1a = LaplacesDemon(Model1_ST, Data_ST1a_100, Initial.Values=c(0,1), Algorithm='CHARM')
MCMC_ST2a = LaplacesDemon(Model2_ST, Data_ST2a_100, Initial.Values=0.5, Algorithm='CHARM')
```

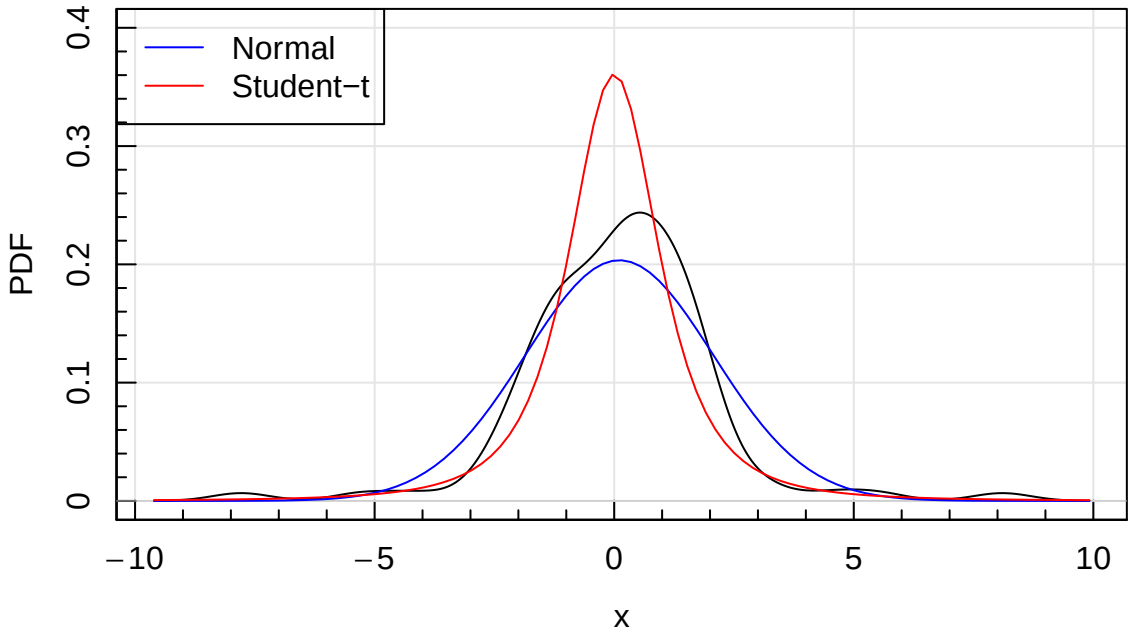
```
magtri(MCMC_ST1a$Posterior2)
```




```
magplot(density(MCMC_ST2a$Posterior2), xlab=expression(lambda), ylab='PDF')
abline(v=log10(3), col='red')
```




```
magplot(density(random_samples), xlab='x', ylab='PDF', ylim=c(0,0.4))
curve(dnorm(x, mean=MCMC_ST1a$Summary2['mean','Mean'], sd=10^MCMC_ST1a$Summary2['sd','Mean']),
      add=TRUE, col='blue')
curve(dt(x, df=10^MCMC_ST2a$Summary2['df','Mean']), add=TRUE, col='red')
legend('topleft', legend=c('Normal','Student-t'), col=c('blue','red'), lty=1)
```



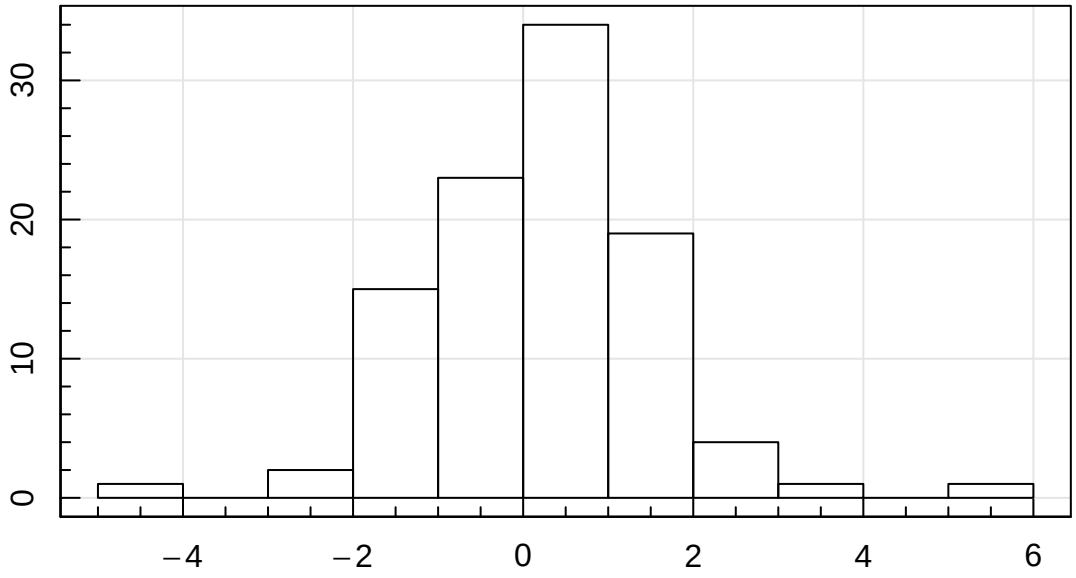

```
BFcalc(list(MCMC_ST1a, MCMC_ST2a))
```

```
## $M_BF
##           [,1]         [,2]
## [1,]     1.00 4.682671e-05
## [2,] 21355.33 1.000000e+00
##
## $post.prob
## [1] 4.682452e-05 9.999532e-01
```

So we strongly prefer the Student-t model here, as we would hope.

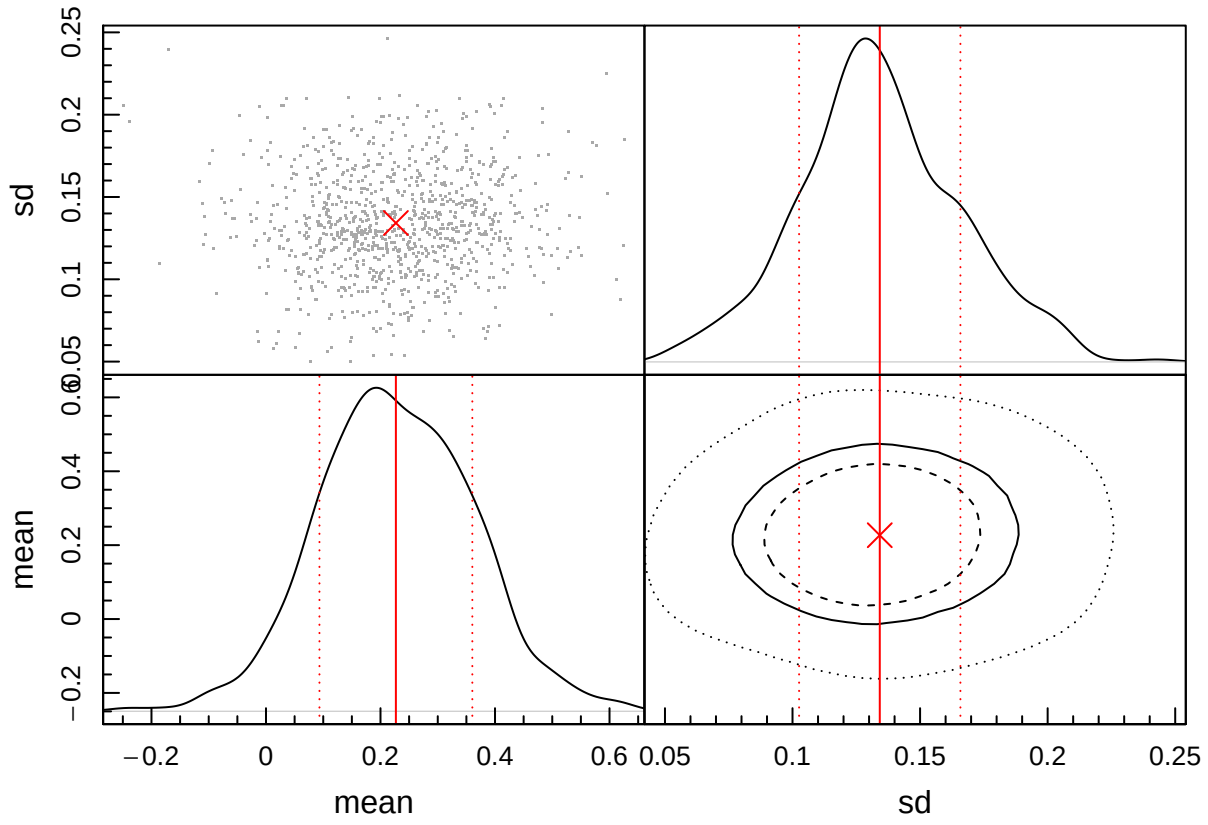
How about if we make the data more "Normal" by increasing the DoF?

```
set.seed(666)
random_samples = rt(100, df=5)
Data_ST1b_1000 = list(data=random_samples, mon.names='', parm.names=c('mean','sd'), N=1e3)
Data_ST2b_1000 = list(data=random_samples, mon.names='', parm.names='df', N=1e3)
maghist(Data_ST1b_1000$data, verbose=FALSE)
```

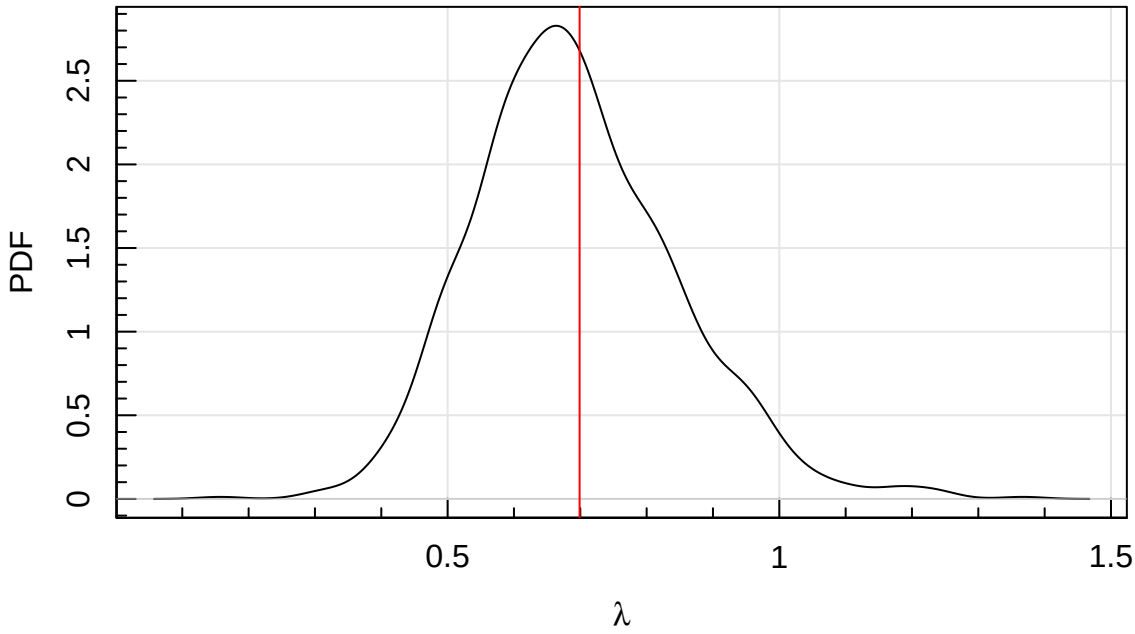



```
MCMC_ST1b = LaplacesDemon(Model1_ST, Data_ST1b_1000, Initial.Values=c(0,0), Algorithm='CHARM')
MCMC_ST2b = LaplacesDemon(Model2_ST, Data_ST2b_1000, Initial.Values=0.5, Algorithm='CHARM')
```

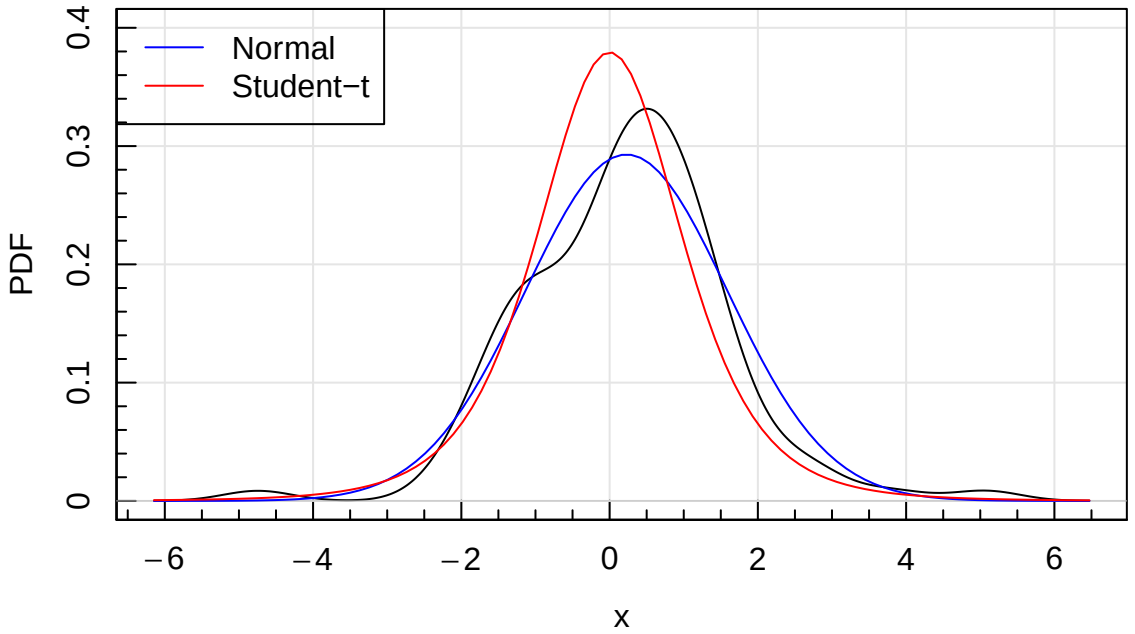
```
magtri(MCMC_ST1b$Posterior2)
```




```
magplot(density(MCMC_ST2b$Posterior2), xlab=expression(lambda), ylab='PDF')
abline(v=log10(5), col='red')
```




```
magplot(density(random_samples), xlab='x', ylab='PDF', ylim=c(0,0.4))
curve(dnorm(x, mean=MCMC_ST1b$Summary2['mean','Mean'], sd=10^MCMC_ST1b$Summary2['sd','Mean']),
      add=TRUE, col='blue')
curve(dt(x, df=10^MCMC_ST2b$Summary2['df','Mean']), add=TRUE, col='red')
legend('topleft', legend=c('Normal','Student-t'), col=c('blue','red'), lty=1)
```




```
BFcalc(list(MCMC_ST1b, MCMC_ST2b))
```

```
## $M_BF
##           [,1]       [,2]
## [1,]  1.00000 0.01628335
## [2,] 61.41243 1.00000000
##
## $post.prob
## [1] 0.01602245 0.98397755
```

In this case we still prefer the Student-t solution (but less certainly), even though the correct DoF parameter is making our data more "Normal".

Model Selection Techniques

So far we have looked at using Bayes Factors to decide between models, but without any explanation of what this represents or any justification of why this might work. There are in fact a large number of different approaches advocated for model selection. Why so many? Well, deep down Bayesian analysis is actually pretty agnostic about determining a preference *between* models, rather it is a tool to infer information about a given model. The fact that it can often help us determine which of multiple models should be preferred should be seen as a bonus when possible, and not a generally reliable assumption.

The reason for the above pessimism is complex in detail (much of statistics is once you get deeper), but in general it is because at least one of our models will be incorrect if we are comparing multiple options, and what the posterior samples inform us about only make sense in the regime of the model being appropriate. More typically, none of the models are correct, e.g. we are knowingly fitting a simplified model to the data in order to extract key information. The latter is nearly always true in astronomy, since if you are trying to build a generative model of a galaxy in order to fit its light profile then just fitting a Sersic profile is clearly incorrect, in detail we would need to realistically evolve a hydrodynamical simulation. The latter is rarely pragmatic when all we really want to know is the half light radius of the galaxy. Clearly in this case a simple Sersic model will be appropriate. However, we are still a bit stuck if we want to rigorously compare a two component model of the galaxy with a bulge and a disk versus a simpler one with just a single component.

The issue is there are a few ways to argue model selection (and these are not all mutually exclusive):

1) The selected model should probably be true (Bayesian)
2) The selected model should best describe the data with least complexity (Information Theory)
3) The selected model should rarely be false (Frequentist)
4) The selected model should be useful and predictive (Pragmatism)

Also in practice when model testing there are at least three regimes we work in:

1) The true model exists within our set of models (and we may or may not know this)
2) The true model does not exist within our set of models (and we may or may not know this)
3) We have one model, and we want to know whether it is 'good' (i.e. we can stop looking for a better model)

Intuitively, if we can achieve a similarly good fit with fewer parameters then this fit should *generally* be preferred. You can think of this as the statisticians application of Occam's razor (Google that if you are unfamiliar). In some sense, this is what many of the most popular model comparison approaches attempt to quantify, since in general the true model should also be the simplest. With our earlier caveats in place, we will look at some of the most popular variants.

Baysian Information Criterion (BIC)

The Bayesian Information Criterion (BIC) is a scheme that aims to select between models. Models with lower BIC are preferred, and it is defined such that:

$$BIC = \ln(n)k - 2\ln(\hat{L}),$$

where n is the number of observations, k is the number of parameters in the model, and $\hat{L}$ is the posterior maximum likelihood value. The argument for using this simple scheme is Bayesian in origin, with the assumption that the posterior is covariant and Normal, and that the prior is well defined given the data.

Akaike Information Criterion (AIC)

Closely related is the Akaike Information Criterion (AIC), which using the same parameter definitions are above can be specified as:

$$AIC = 2k - 2\ln(\hat{L}).$$

Again, lower AIC value models should be preferred. The penalty for adding parameters to your model is in general less in AIC than BIC (assuming you have more than 7 observed data points). This can mean that BIC favours overly simple models for small n, but it will almost certainly return the true model as n approaches infinity (which AIC will not for certain).

Watanabe–Akaike/Baysian Criterion (WAIC and WBIC)

Related to the above are the Watanabe–Akaike/Bayesian Criterion (WAIC and WBIC). These basically replace the maximum likelihood $\hat{L}$ above with the log posterior predictive density (LPPD). The LPPD can be computed as follows:

- For each observation x, compute the likelihood of observing this result for all sampled posterior parameters (so the burnt in thinned samples).
- Compute the mean of L for each x.
- Take the natural log of this mean for each x.
- Sum the log of the mean likelihoods across all observations in x.

It might be possible to keep track of the likelihood information required during the MCMC run (giving an $n \times N_s$ matrix) where N_s is the number of kept samples (or iterations), so typically 10,000s. Otherwise, this will need to be computed after the fact. Once the LPPD has been computed the WBIC and WAIC then become:

$$WBIC = \ln(n)k - 2.LPPD,$$

$$WAIC = 2k - 2.LPPD,$$

with k and n having the same meaning as above.

Deviance Information Criterion (DIC)

The Deviance Information Criterion (DIC) is closely related to the AIC, but it uses more information about the posterior samples. Namely we define:

$$D(\theta) = -2LL(\theta)$$

Where $LL(\theta)$ is our summed log-likelihood of the data given the model for a particular parameter set sample θ that we monitor explicitly in **LD** (and from an earlier chapter, we can see this actually the χ^2 ignoring the offset, which is already tracked in **LaplacesDemon**). The mean of this quantity we define as $D(\bar{\theta})$.

Next we define the effective number of parameters (also known as the model complexity) as:

$$p_D = \text{var}(D(\theta))$$

Finally the DIC is:

$$DIC = 2p_D - D(\bar{\theta}).$$

The DIC is usually a good compromise solution, since it adapts the number of parameters to the number of effective parameters, which is an interesting quantity in itself. E.g. in our example above we tried to fit the Student-t with a Normal with a mean and standard deviation. Since the posterior plots do not show much correlation, this would suggest our effective number of parameters is indeed quite close to 2, but we can quantify this using the DIC output computed by **LaplacesDemon** (second element of *DIC2*):

```
MCMC_ST1b$DIC2[2]
```

```
## [1] 2.212
```

This is actually larger than 2, but in cases where the parameters are highly correlated it will generally be a number less than k. This makes sense, because the more parameters can 'predict' each other the less fundamental freedom can exist. Consider perfectly correlated parameters- knowing one would immediately inform you of the value of the other. In these cases there is usually an implicit hidden parameter that is driving both, and ideally we fit using this intrinsic parameter instead. For convenience reasons, many distributions have known degenerate parameter dependence- e.g. the M^* and α parameters of the Schechter distribution function often used in astronomy for number counts. It is possible to parametrise this so the free parameters are orthogonal, but they are then less human interpretable (which does matter too).